Learning Subjective Query Concepts and Measuring Perceptual Similarity in High Dimensional Spaces

Edward Chang and Beitao Li

Department of Electrical and Computer Engineering
University of California, Santa Barbara, CA 93106
echang@ece.ucsb.edu, beitao@engineering.ucsb.edu

Abstract

The proliferation of digital cameras and the heightening of national security have intensified the need for effective, efficient image analysis and retrieval. In this paper, we address two central database issues regarding image retrieval: formulating image-query concepts, and measuring perceptual similarity. We present the preliminary results of our research and discuss future research directions.

1 Introduction

Research in image retrieval has gained steady momentum in recent years as a result of the dramatic increase in the volume of digital images and video. However, two important database problems remain largely unsolved: how best to formulate users' subjective image-query concepts, and how to accurately measure perceptual similarity.

1. *Formulating an image query concept.* For relational database searches, query concepts are conveyed via query languages. For an image search task, it is difficult for a user to describe a desired image using low-level features such as color, shape and texture (these are widely used features for representing images [69]). In addition, perception is subjective, and so is the way in which individuals may articulate a query concept. Many methods have been proposed to learn users' subjective query concepts through machine learning and relevance feedback. However, traditional techniques are ineffective for online query-concept learning for at least two reasons.

- *Time and sample constraints*. Traditional learning methods such as decision trees and neural networks require a large number of training instances (i.e., samples) and can take a long time (more than a few seconds) to learn a concept [45, 55]. But, online users are typically impatient and cannot be expected to wait around for results or to provide a great deal of feedback during the search.
- *Seeding constraint*. All traditional relevance feedback methods [42, 63, 86] require users to provide "good" examples to seed a query. However, finding good seeds is the job of the search engine itself, and this circular requirement leaves the core problem— learning users' query concepts—unsolved.

2. *Measuring Perceptual Similarity*. Once a query concept is learned, images that are *similar* to the concept are retrieved. To achieve effective retrieval, an image system must be able to accurately characterize and quantify perceptual similarity. Various distance functions, such as Minkowski metric, earth mover distance, and fuzzy logic, have been used to measure similarity between feature vectors representing images. However, our experiments show that these functions often cannot find images that are obviously similar, and hence are not adequate for measuring perceptual similarity.

In this paper, we present our recent work in addressing the above problems. For formulating subjective query concepts, we present two base learning algorithms, MEGA [13] and SVM_{Active} [79], that we have developed, and then propose hybrid learning schemes for further improving the performance of these base algorithms. For measuring image similarity, we present a perceptual distance function—Dynamic Partial Function (DPF)—that we have discovered through mining a very large set of visual data. We show that DPF empirically outperforms Minkowski-like distance functions, including the recently proposed fractional distance function [2]. More significantly, we show the crucial link between DPF and similarity theories in cognitive science.

Illustrative Example

An example query is presented here to show how our learning algorithms learns a query concept. This interactive query session involves three rounds of user iterations that are illustrated in four figures. The user's query concept in this example is "wild animals." Our online prototype is available at [11, 12].

Initial Screen. The search engine presents the initial screen to the user as depicted in Figure 16 (in Appendix). The screen is split into two frames vertically. On the left-hand

side of the screen is the learner frame; on the right-hand side is the result frame. Through the learner frame, the search engine learns what the user wants via an intelligent sampling process. The result frame displays what the search engine thinks the user wants. (The user can set the number of images to be displayed in these frames.)

Iteration $\#1$. Sampling and relevance feedback starts. Once the user clicks the "submit" button in the initial frame, the sampling and relevance feedback step commences to learn what the user wants. The search engine presents a number of samples in the learner frame, and the user highlights images that are relevant to her query concept by clicking on the relevant images. As shown in Figure 17, three images (the third image in rows one, two and four in the learner frame) are selected as relevant, and the rest of the unmarked images are considered irrelevant. The user indicates the end of her selection by clicking on the submit button in the learner frame. This action brings up the next screen.

Iteration $\#2$. Sampling and relevance feedback continues. Figure 18 shows the third screen. First, the result frame displays what the search engine thinks will match the user's query concept at this time. As the figure indicates, the top nine returned images fit the concept of "wild animals." The user's query concept has been captured, though somewhat fuzzily. The user can ask the search engine to further refine the target concept by selecting relevant images in the learner frame. In this example, the fourth image in the second row and the third image in the fourth row are selected as relevant to the concept. After the user clicks on the submit button in the learner frame, the next screen is displayed.

Iteration $\#3$. Sampling and relevance feedback continues. The result frame in Figure 19 shows that ten out of the top twelve images returned match the "wild animals" concept. The user selects four relevant images displayed in the learner frame. This leads to the final screen of this learning series.

Final Result. Sampling and relevance feedback ends. Figure 20 shows that all returned images in the result frame fit the query concept (wild animals).

The example shows three major features that are crucial for effective and efficient image retrieval:

1. *Eliminating the seeding requirement*. Our learning system does not require a query to be initiated with "good" seed images. Even if all initial samples presented to the user are negative, the negative-labeled instances can be used to shrink the pool of the next sampling round substantially. Therefore, the probability of obtaining a relevant example in the next iteration will be substantially higher.

2. *Supporting flexible concept formulation*. One can search for a general animal concept or a specific kind of animal (e.g., tigers). For example, if we mark only tiger images as relevant, the final result would show tiger images only. Also note in the second screen that the search engine presents animals of different kinds, and in different shapes, and on different backgrounds to ask for feedback. The learner can intelligently explore more possibilities to refine the query concept.

3. *Accomplishing the above tasks quickly and accurately*.

Although early results show definite promise, many research challenges remain. Regarding learning, for instance, a very specific query concept is difficult to learn because an image relevant to that concept is hard to find. Multimodal approaches and new statistical models are needed to conduct inferences under these kinds of extreme constraints. Regarding similarity searches, for instance, we require new indexing methods for the dynamic and non-metric natures of the perceptual distance function that we derived. We enumerate several research challenges and our research plans in Section 7.

The rest of the paper is organized as follows: Section 2 discusses related work. Section 3 presents MEGA, SVM_{Active}, and hybrid learning schemes. We present experimental results of comparing these learning methods in Section 4. Section 5 depicts our mining effort for finding DPF and its link to similarity theories in cognitive psychology. Section 6 shows that DPF is more effective than traditional functions through empirical studies. Finally, we discuss research challenges in Section 7.

2 Related Work

As mentioned in Section 1, the traditional methods for learning and measuring perceptual similarity can be considerably improved. This section analyzes the limitations of the existing work.

2.1 Related Work in Learning Query Concepts

Machine learning and relevance feedback techniques have been proposed to learn and to refine query concepts. The problem is that most traditional techniques require a large number of training instances [3, 45, 55, 88], and they require seeding a query with "good" examples [42, 63, 86]. Unfortunately, in many practical scenarios, a learning algorithm must work with a scarcity of training data and a limited amount of training time.

2.1.1 Machine Learning

Of late, ensemble techniques such as *bagging* [5], *arcing* [6], and *boosting* [30, 71] have been proposed to improve learning accuracy for decision trees and neural networks. These ensemble schemes enjoy success in improving classification accuracy through bias or variance reduction, but they do not help reduce the number of samples and time required to learn a query concept. In fact, most ensemble schemes actually increase learning time because they introduce learning redundancy in order to improve prediction accuracy [22, 30, 39, 56].

To reduce the number of required samples, researchers in the machine learning community have conducted several studies of active learning [16, 10] for classification. Active learning can be modeled formally as follows: Given a

dataset S consisting of an unlabeled subset U and a labeled subset X, an active learner L has two components: f and s. The f component is a classifier that is trained on the current set of labeled data X. The second component s is the sampling function that, given a current labeled set X, decides which subset u in U to select to query the user. The active learner returns a new f after each round of relevance feedback. The sampling techniques employed by the active learner determine the selection of the next batch of unlabeled instances to be labeled by the user.

The *query by committee* (QBC) algorithm [26, 72] is a representative active learning scheme. QBC uses a distribution over all possible classifiers and attempts greedily to reduce the entropy of this distribution. This general purpose algorithm has been applied in a number of domains (although, to our knowledge, not to the image retrieval domain) using classifiers (such as Naive Bayes classifiers [21, 52]) for which specifying and sampling classifiers from a distribution is natural. Probabilistic models such as the Naive Bayes classifier provide interpretable results and principled ways to incorporate prior knowledge. However, they typically do not perform as well as discriminative methods such as SVMs [40, 24], especially when the amount of training data is scarce. (See our SVM-based approach in Section 3.1.2.) For image retrieval where a query concept is typical non-linear[1], our MEGA and SVM_{Active} with kernel mapping provide more flexible and accurate concept modeling.

Specifically for image retrieval, the PicHunter system [19, 18, 17, 20] uses Bayesian prediction to infer the goal image, based upon users' input. Mathematically, the goal of PicHunter is to find a single goal point in the feature space (e.g., a particular flower image), whereas our goal is to hunt down all points that match a query concept (e.g., the entire flower category, which consists of flowers of different colors, shapes, and textures, and against different backgrounds). Note that the points matching a target concept can be scattered all over the feature space. To find these points quickly with few hints, our learning algorithms must deal with many daunting challenges.

2.1.2 Relevance Feedback

The study of [64] puts relevance feedback techniques proposed by the Information Retrieval (IR) into three categories: *query reweighting*, *query point movement* and *query expansion*.

- *Query reweighting* and *query point movement* [38, 59, 58, 63, 70]. Both query reweighting and query point movement use nearest-neighbor sampling: They return top ranked objects to be examined by the user and then refine the query based on the user's feedback. If the initial query example is good and the query concept is convex in the feature space [38, 86], this nearest-neighbor sampling approach works fine. Unfortunately, most users do not have a good example to start a query, and most image-query concepts are non-convex.
- *Query expansion* [64, 86]. The *query expansion* approach can be regarded as a multiple-instances sampling approach. The samples of a subsequent round are selected from the neighborhood (not necessarily the nearest ones) of the positive-labeled instances of the previous round. The study of [64] shows that query expansion achieves only a slim margin of improvement (about 10% in precision/recall) over query point movement.

All traditional relevance feedback methods require seeding the methods with "good" positive examples [25, 31, 34, 51, 74, 75, 84], and most methods do not use negative-labeled instances effectively. For instance, sunset images must be supplied as examples in order to search for sunset pictures. As we stated in Section 1, finding good examples is the job of a search engine itself. Our learning methods effectively use negative-labeled instances to corner the positive instances, and select the most informative unlabeled instances to query user feedback. Because of the effective use of negative and unlabeled instances, our method can learn a query concept much faster and more accurately than the traditional relevance-feedback methods.

2.2 Related Work in Measuring Perceptual Similarity

Extensive work has been done on mapping color spectrums and textures to the perceptual space [27, 50, 78, 82, 87]. Until recently, little has been done on finding a good perceptual distance function for aggregating individual perceptual features (work that has been done will be discussed shortly). Most researchers quantify the difference between two feature vectors by their Minkowski distance or its variants. The advantages of employing the Minkowski distance are 1) similarity characterization is simple, and 2) similarity searches can be supported by traditional indexing algorithms. However, in [76], it was shown that for image retrieval the Minkowski distance results in many false negatives and thus yields low recall.

Rubner et al. [67] propose Earth Mover's Distance (EMD) for image retrieval and claim marginal improvement (about 10 percentile in recall) over the Minkowski distances. However, their results may not be conclusive, since only a few queries were tested, and those were on small datasets.

The weighted Minkowski function introduces feature weights into the distance function [64, 68]. The feature weights are usually obtained through a user's feedback indicating how relevant the results are to the query. The motivation is to identify important features by refining the weighting vector. The shortcoming of this metric is that images similar to the query image cannot be captured by just one static set of weights. For instance, both a down-sampled and a cropped image are perceptually similar to the original image. But the similarity of the down-sampled image is primarily captured in the color features (texture features are changed when an image is down-sampled), whereas the similarity of the cropped image may be captured in the texture features (color histograms are changed when an image is cropped). Using a static feature subset to find both down-sampled and cropped images will not work effectively

[1] A query such as "animals", "women", and "european architecture" does not reside contiguously in the space formed by the image features.

because it cannot adapt to such changes.

Some other distance functions such as the quadratic-form distances proposed by [25, 38], have been shown to perform marginally better than the Minkowski distance function. In fact, the quadratic-form distance functions are variants of the weighted Euclidean distance function, but they operate in a transformed feature space. Thus they still cannot overcome the limitations of static weighting and do not significantly improve the retrieval performance. Recently, the fractional distance function [2] has been shown to outperform both L1 and L2 distance functions. Our work shows that even better functions can be derived, and the function that we derive (presented in Section 5) not only works well but can also be interpreted by perceptual principles in cognitive science.

Feature reduction has been shown to be a useful method for improving the quality of similarity searches in the text domain. The data is transformed to a lower-dimensional space where most information is preserved, by using methods such as Principle Component Analysis (PCA). Although PCA-like feature reduction has been widely used, the work of C. Aggarwal [1] points out that the effectiveness of feature reduction can be domain-dependent and data-dependent. Moreover, for image retrieval, we have not found PCA very useful for two reasons. First, the perceptual features employed to characterize images (such as color, texture, and shape) are sufficiently independent of one another. This agrees with C. Aggarwal's observation that when the dimensions are relatively uncorrelated to one another, dimension reduction can be counter-productive. Second, and more importantly, image perception is subjective, and hence feature reduction in the image domain must be conducted in a user-dependent way, not in a data-dependent way. Our approach is first to use learning methods such as MEGA and SVM$_{Active}$ to perform implicit feature transformation in a user-dependent and query-dependent manner. After a more effective feature space is found, our dynamic partial distance function (DPF) performs dynamic feature reduction for each pair of images. The similarity of different pairs of images is measured in different subspaces. We explain in Section 5 why this method works effectively both empirically and theoretically.

3 Learning Query Concepts

Since the traditional machine learning and relevance feedback methods may not be adequate for effective online concept learning, how can they be improved? In this section, we first present our preliminary research results to set up a context (Section 3.1). We then present our hybrid learning schemes (Section 3.2).

3.1 Base Algorithms

Here, we describe MEGA and SVM$_{Active}$, two online learning algorithms that we have recently developed[2] MEGA is an inductive-based active learning algorithm; SVM$_{Active}$ is a margin-based active learning algorithm. For detailed algorithms and extensive experimental results, please refer to [13, 79].

3.1.1 MEGA

The Maximizing Expected Generalization Algorithm (MEGA) models query concepts in k-CNF [44], which can formulate virtually all practical query concepts[3]. MEGA uses k-DNF to bound the sampling space from which to select the most informative samples to solicit user feedback.

Definition 1: k-CNF: For constant k, the representation class k-CNF consists of Boolean formulae of the form $c_1 \wedge \cdots \wedge c_\theta$, where each c_i is a disjunction of at most k literals over the Boolean variables $x_1, \ldots, x_n$. (A Boolean variable represents an image feature.) No prior bound is placed on θ.

Definition 2: k-DNF: For constant k, the representation class k-DNF consists of Boolean formulae of the form $d_1 \vee \cdots \vee d_\theta$, where each d_i is a conjunction of at most k literals over the Boolean variables $x_1, \ldots, x_n$. No prior bound is placed on θ.

MEGA initializes the query concept-space (QCS) as a k-CNF and the candidate concept-space (CCS) as a k-DNF. The QCS starts as the most specific concept and the CCS as the most general concept. The target concept that the learner learns is more general than the initial QCS and more specific than the initial CCS. The learner learns the QCS, while at the same time refining the CCS to delimit the boundary of the sampling space. (The shaded area in Figure 1 shows the sampling space that is between the QCS and the CCS.)

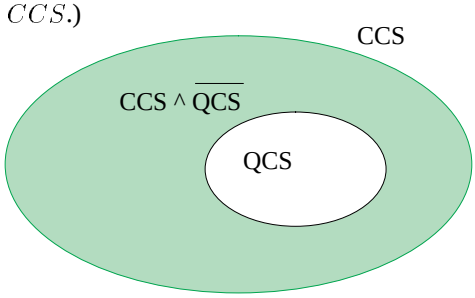


Figure 1: MEGA's Sampling Space: $CCS \wedge \overline{QCS}$.

Intuitively, at a given stage, we have a QCS and a CCS, representing the boundary of the candidate concept-space. To make sure that we choose the most useful examples, we have tested two strategies.

1. Bounding the sample space: Avoid choosing useless unlabeled instances by using the CCS and QCS to delimit the sampling boundary.

2. Maximizing the usefulness of a sample: Choose an example that shall remove the maximum expected number of disjunctive terms from QCS. In other words, we choose an example that can maximize the expected generalization

[2] This paper does not present feature-extraction techniques. We organize useful features (e.g., [9, 32, 33, 35, 49, 50, 61, 62, 67, 73, 77, 85]) in a hierarchical/multi-resolution manner to assist effective and efficient learning [13, 79].

[3] k-CNF is more expressive than k-term DNF, and it has both polynomial sample complexity and time complexity [45, 54].

of the concept. If the example is labeled negative by the user, it can be useful to remove conjunctive terms in the CCS.

It may appear that if we pick an example that has more dissimilar disjunctions (compared to the QCS), we may have a better chance of eliminating more disjunctive terms. This is, however, not true. An example must be labeled by the user as positive to be useful for refining QCS. Unfortunately, an example with many disjunctions that are dissimilar to the target concept is less likely to be labeled positive. Therefore, there is a tradeoff between choosing an example that has more contradictory terms and choosing an example that is more likely to be labeled positive.

Let Ψ denote the number of disjunctions remaining in the concept. The number of disjunctions that can be eliminated in the current round of sampling, denoted as ψ, is between zero and Ψ. We can write the probability of eliminating ψ terms as $P_e(\psi)$. $P_e(\psi)$ is a monotonically decreasing function of ψ. We attempt to find the ψ that can eliminate the maximum expected number of disjunctive terms, given an example. Our objective function can be written as

$$\psi^* = arg_\psi \max E(\psi) = \psi \times P_e(\psi). \quad (1)$$

To solve ψ^*, we must know $P_e(\psi)$, which can be estimated by the two methods below: probabilistic estimation and empirical estimation. (Please refer to [13] for detailed discussion.) We have compared MEGA with the five traditional sampling schemes: *random*, *bounded random*, *nearest neighbor*, *query expansion* and *aggressive*. Our preliminary experimental results show that when the number of labeled instances is limited, MEGA substantially outperforms traditional sampling schemes in learning query concepts.

3.1.2 SVM$_{Active}$

Once some relevant and some irrelevant samples are marked, we can employ SVM$_{Active}$ to refine the class boundary. SVMs have strong theoretical foundations and excellent empirical successes. They have been applied to tasks such as handwritten digit recognition [81], object recognition [60], and text classification [40]. We added the active learning component to SVMs for selecting the most informative samples to query a user and to quickly refine a boundary that separates relevant data objects from the rest of the dataset.

For the purpose of query-concept learning, we consider SVMs in the binary classification setting. We are given training data $\{\mathbf{x_1} \ldots \mathbf{x_n}\}$ that are vectors in some space $\mathcal{X} \subseteq \mathbb{R}^\ell$. We are also given their labels $\{y_1 \ldots y_n\}$ where $y_i \in \{-1, 1\}$. In their simplest form, SVMs are hyperplanes that separate the training data by a maximal margin. All vectors lying on one side of the hyperplane are labeled as -1, and all vectors lying on the other side are labeled as 1. The training instances that lie closest to the hyperplane are called *support vectors*. More generally, SVMs allow us to project the original training data in space $\mathcal{X}$ to a higher dimensional feature space $\mathcal{F}$ via a Mercer kernel operator K. In other words, we consider the set of classifiers of the form: $f(\mathbf{x}) = \sum_{\mathbf{i=1}}^{\mathbf{n}} \alpha_\mathbf{i} \mathbf{K}(\mathbf{x_i}, \mathbf{x})$. When $f(\mathbf{x}) \geq \mathbf{0}$ we classify $\mathbf{x}$ as $+1$, otherwise we classify $\mathbf{x}$ as -1.

When K satisfies Mercer's condition [8] we can write: $K(\mathbf{u}, \mathbf{v}) = \mathbf{\Phi}(\mathbf{u}) \cdot \mathbf{\Phi}(\mathbf{v})$ where $\Phi : \mathcal{X} \rightarrow \mathcal{F}$ and "$\cdot$" denotes an inner product. We can then rewrite f as:

$$f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{\Phi}(\mathbf{x}), \text{ where } \mathbf{w} = \sum_{\mathbf{i=1}}^{\mathbf{n}} \alpha_\mathbf{i} \mathbf{\Phi}(\mathbf{x_i}). \quad (2)$$

Thus, by using K we are implicitly projecting the training data into a different (often higher dimensional) feature space $\mathcal{F}$. The SVM then computes the α_is that correspond to the maximal margin hyperplane in $\mathcal{F}$. By choosing different kernel functions, we can implicitly project the training data from $\mathcal{X}$ into spaces $\mathcal{F}$ for which hyperplanes in $\mathcal{F}$ correspond to more complex decision boundaries in the original space $\mathcal{X}$.

Intuitively, SVM$_{Active}$ works by combining the following three ideas:

1. SVM$_{Active}$ regards the task of learning a target concept as one of learning an SVM binary classifier. An SVM captures the query concept by separating the relevant images from the irrelevant images with a hyperplane in a projected space, usually a very high-dimensional one. The projected points on one side of the hyperplane are considered relevant to the query concept and the rest irrelevant.

2. SVM$_{Active}$ learns the classifier quickly via active learning. The active part of SVM$_{Active}$ selects the most informative instances with which to train the SVM classifier. This step ensures fast convergence to the query concept in a small number of feedback rounds.

3. Once the classifier is trained, SVM$_{Active}$ returns the top-k most relevant images. These are the k images farthest from the hyperplane on the query concept side.

3.2 Hybrid Algorithms

Although MEGA and SVM$_{Active}$ are effective active learning algorithms, we believe combining their strengths can result in even better learning algorithms. In this section we propose three such hybrid algorithms:

- Pipeline learning (Section 3.2.1),
- Transductive learning (Section 3.2.2), and
- Recursive committee and subspace co-training (Section 3.2.3).

3.2.1 Pipeline Learning

As mentioned in Sections 1 and 3.1, learning a query concept faces three challenging steps:

1. *Initialization*. The learner must quickly identify some positive and some negative examples to obtain a fuzzy boundary separating the objects relevant to the query concept and the others.

2. *Refinement*. The learner must then refine the classification boundary with a minimum number of labeled in-

stances (i.e., with minimum rounds of relevance feedback from users).

3. *Ranking*. When the binary classifier has been learned with sufficient confidence, the learner must quickly return the top-k objects that are relevant to the query concept.

As depicted in Section 3.1.2, the SVM$_{Active}$ scheme needs at least one positive and one negative example to start. MEGA is not restricted by this seeding requirement, and it is able to increase the probability for finding relevant examples by refining the sampling boundary using negative-labeled instances. It is therefore logical to employ MEGA to perform the *initialization* task. After some relevant images have been found, the refinement step can be executed by either MEGA or SVM$_{Active}$.

Thus, we can have three execution alternatives.

1. MEGA *only*. Use MEGA all the way to learn a concept.

2. SVM$_{Active}$ *only*. Use random sampling to find the first relevant example(s) and then use SVM$_{Active}$ to learn a concept.

3. *Pipeline learning*. Use MEGA to find initial relevant objects, and then switch to SVM$_{Active}$ for refining the binary classifier and ranking returned objects.

3.2.2 Transductive Learning

The major challenge that our learning task faces is the scarcity of training data. As counter-intuitive as this may sound, the scarcity of negative examples is more problematic than the scarcity of positive examples for our learning algorithms. This is because describing a concept such as "tigers" may be challenging, but describing a negated concept (in this case the "non-tiger" concept) can require potentially infinite information. (The number of images needed to adequately portrait the tiger concept is finite, but the number of non-tiger images is infinite.) Negative examples are significantly easier to come by, but at the same time, the number of negative samples needed to depict a negated concept can be very large. Thus, we need a substantial number of negative examples to accurately characterize the class boundary.

Let us revisit the SVM$_{Active}$ algorithm. At each round of sampling, it selects objects that are in the margin and also close to the dividing hyperplane. If the margin is wide (because of the lack of support at the negative-example side), the probability is low that a positive example can be found in the wide margin. Our goal is to find enough positive examples to depict the target concept. However, the difficulty in gathering sufficient negative examples slows down the progress for finding positive examples. The learning process is analogous to mining gold in a stream. The most productive way to harvest gold (i.e., relevant instances) is to quickly filter out water, mud and sand (i.e., negative-labeled instances).

For solving the problem of insufficient training data, transductive learning has been proposed to work with different learning algorithms for various applications [29, 41]. The basic idea of transduction is to leverage the unlabeled data near the labeled data. Suppose the nearest neighbors of negative examples are always negative. We can then use the neighboring examples to substantially increase the negative-labeled pool. However, transduction must be performed with care, since the mis-labeling of data caused by wrong "guesses" can degrade the learning result.

The performance of transduction is, unfortunately, consistently inconsistent. Its performance could not only be application-dependent, but also dataset-dependent. For image retrieval, transduction may not be suitable for the following two reasons:

- Nearest-neighbor sampling. Most image query concepts are not convex, and nor do they reside continuously in the feature space. For instance, flowers of different colors tend to spread out in the color-feature dimension. Thus, to describe the non-flower concept, we need to explore the feature space more aggressively. Suppose a picture of lake (one of many non-flower instances) is labeled negative. Using the neighboring pictures of this lake picture, which tend to be lake-like or something visually similar, may not be very productive in improving the description of the negated concept.
- Noise factor. Again, mislabeled data can reduce the prediction accuracy of the classifier.

3.2.3 Recursive Committee and Subspace Co-training

The problem of using a large unlabeled sample pool to boost performance of a learning algorithm is considered under the framework of co-training [4]. A broad definition of co-training is to provide for each instance multiple distinct views. We employ co-training to find more diversified negative instances than those the nearest-neighbor transductive learning tends to produce. We propose two co-training schemes: *committee co-training* and *subspace co-training*.

Committee Co-training

To find useful negative-labeled instances, we first propose a "committee co-training" scheme by employing MEGA and SVM$_{Active}$ as the committee members to aid a more explorative process. The presence of two distinct views of each example suggests strategies in which two learning algorithms can be trained separately in each view, and then each algorithm's predictions for new unlabeled instances are used to enlarge the training set of the other. We propose the following committee co-training steps:

1. Use SVM$_{Active}$ to select samples and present them to the user for feedback.

2. Use MEGA and the negative-labeled instances to refine CCS (the k-DNF that bounds the sampling space of MEGA).

3. As discussed in Section 3.1.1, the concepts that are more general than CCS do not satisfy the target concept. All training data outside of the MEGA's sampling boundary can thus be labeled negative. They can be used to feed SVM$_{Active}$ for refining the hyperplane.

4. Go back to Step 1 until no more negative-labeled instance can be inferred.

Recursive Subspace Co-training

We have just shown that distinct views can be provided by different learning algorithms. Here, we propose to provide each training instance distinct views via subspace learning. This method recursively conducts subspace co-training at each feedback iteration in the following steps:

1. Divide the feature space into G subspaces.
2. Conduct parallel training in these G subspaces using labeled training dataset L.
3. Use the G resulting learners to label the unlabeled pool and yield a new set of labeled instances L'.
4. $L \leftarrow L \cup L'$
5. Go back to Step 1 until no more negative-labeled instance can be inferred (i.e., until $L' = \emptyset$).

4 Empirical Study I - Learning Algorithms

We implemented both MEGA and SVM$_{Active}$ in C, $C++$, and tested using an Intel Pentium III^{TM} workstation running Linux. We have implemented an industrial strength prototype [11, 12] with all features discussed in this paper. We have shown in [13, 79] that MEGA and SVM$_{Active}$ work substantially better than the traditional schemes. In this study, we tested our prototype with the intent of answering three central questions.

1. Does using MEGA to find the first relevant object(s) and then switching to SVM$_{Active}$ a more effective algorithm than using the two learning algorithms individually?
2. Does the percentage of relevant data affect learning performance? (If we reduce the matching data from 5% of the dataset to 1%, does it take more iterations to learn query concepts?)
3. Do additional heuristics such as transduction and co-training help accelerate concept convergence?

4.1 Data

For our empirical evaluation of our learning methods we used a real-world image dataset: a twenty-category image dataset where each category consisted of 200 to 300 images. Images for this dataset were collected from Corel Image CDs The set consists of images of *architecture*, *bears*, *cities*, *clouds*, *couples*, *flowers*, *insects*, *ladies*, *landscape*, *objectionable images*, *planets*, *tigers*, *tools*, *waves*, *elephants*, *fabrics*, *fireworks*, *food*, and *textures*.

To enable an objective measure of performance, we assumed that a query concept was an image category, or a superset or a subset of a category. For instance, a query can be "wild animals" including "tigers", "elephants", and "bears." A query can be "white bears" (a subset of "bears"), or "purple flowers" (a subset of "flowers"). Each image is represented by a 144-dimension vector [79]. The learners do not have any prior knowledge about image categories.

4.2 Queries

We separate our queries into two categories: 1% and 5% queries. The matching images for each of the more specific query concepts such as "purple flowers" and "white bears" account for about 1% of the total dataset. More general concepts such as "bears", "flowers" and "architectures" have about 5% matching images in the dataset. For each experiment, we will report results for these two categories of queries separately. To measure accuracy, we report precisions of the top-10 and top-20 retrievals. We choose to report top-k precisions because top-k queries fit online image-retrieval scenarios naturally [14], and precision is the best measure for the accuracy of a top-k query. (A system that achieves high recall does not necessarily return high quality results in the top-k pool.) In each user iteration, our system presents sixteen images to solicit user feedback. For each test case, we ran the test 100 times and tallied the average precision.

4.3 Results and Discussion

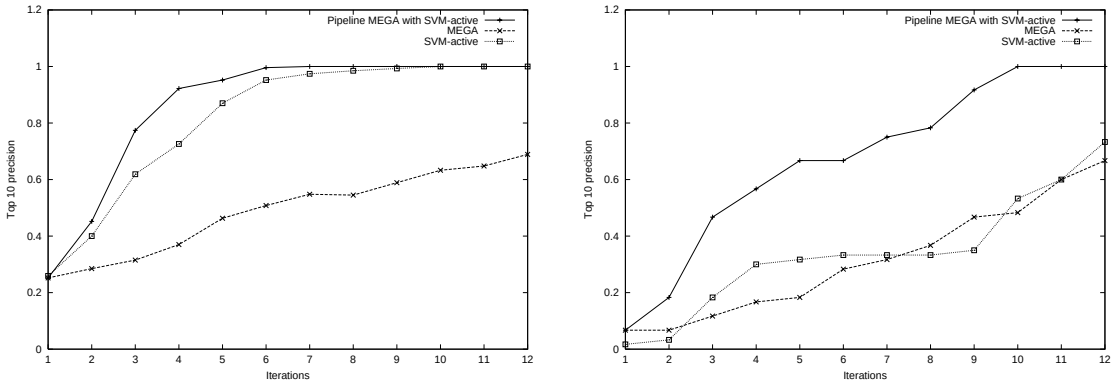
Here we present our analysis of the results, organized with regard to the central questions listed at the start of Section 4.

4.3.1 MEGA, SVM$_{Active}$, and Pipelining

For top-10 retrieval with 5% matching data (Figure 2(a)), SVM$_{Active}$ clearly outperforms MEGA. The major weakness of SVM$_{Active}$ is in initialization—finding the first few positive samples. Such weakness may not seriously affect this experiment, since there is a high probability of finding one of the 5% positive examples through random sampling. For queries with only 1% matching data (Figure 2(b)), such a weakness becomes more significant, because it substantially degrades SVM$_{Active}$'s performance. Especially, its precision in the first two iterations is very low. Overall, the precision of MEGA and SVM$_{Active}$ is similar for 1% queries.

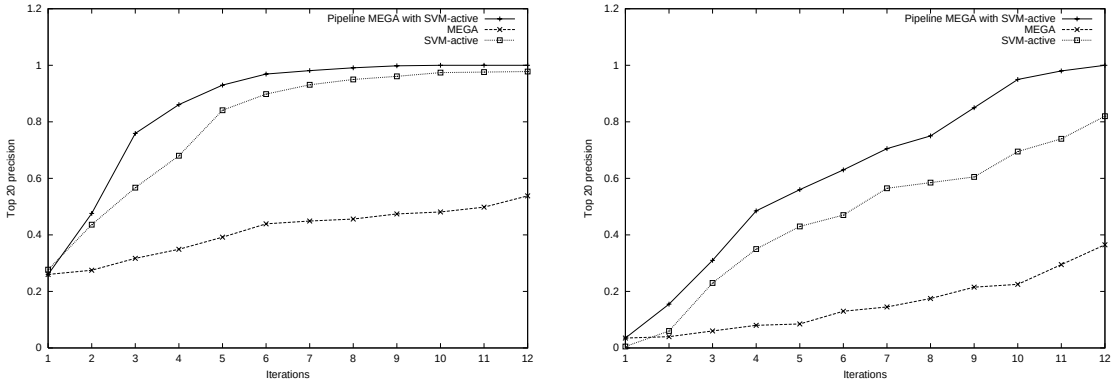
The hybrid algorithm (pipelining MEGA with SVM$_{Active}$) clearly outperforms SVM$_{Active}$ and MEGA when they are used individually. The difference in precision is more significant for queries with 1% matching data than for those with 5% matching data. This trend indicates the strength of the hybrid algorithm in handling more specific query concepts and/or larger datasets. Note that the precision of the hybrid algorithm reaches 80% after 3 and 6 iterations, respectively, for 5% and 1% experiments (Figures 2(a) and (b)). It takes 6 and 10 iterations, respectively, for the algorithm to achieve near-100% precision.

For top-20 retrieval (Figures 3(a) and (b)), the precision of the hybrid algorithm remains the highest, followed by SVM$_{Active}$ and MEGA. As expected, the differences in their performances are more significant for queries with 1% matching data than for those with 5% matching data. For 1%



(a) (b)

Figure 2: Precision versus iterations for top-10 retrieval (a) 5% and (b) 1% queries.



(a) (b)

Figure 3: Precision versus iterations for top-20 retrieval (a) 5% and (b) 1% queries.

queries, the hybrid algorithm achieves near-100% precision after 11 iterations.

4.3.2 Transductive Learning

This experiment examined the effect of transductive learning on the image dataset. We divided our investigation into two parts:

- The noise factor of the dataset.
- Transduction on negative examples.

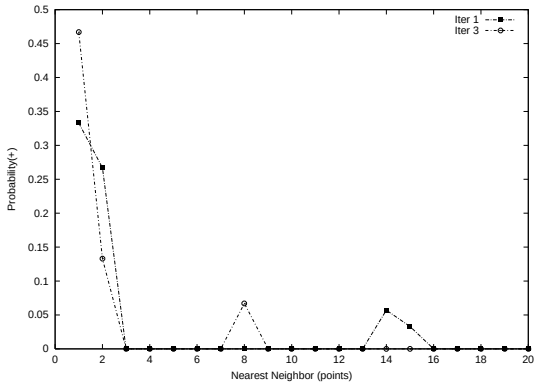


Figure 4: Noisy Positive Neighbors.

Noise of the Dataset

We first examined the effectiveness of using positive-labeled instances to perform transduction. Figure 4 shows the average probability of the top 20 nearest neighbors being positive for a positive-labeled instance, after the first and third feedback iterations, respectively. For example, after three iterations, the probability that the nearest neighbor of a positive-labeled instance will be positive is less than 50%. The chance of the second nearest neighbor being positive is less than 15%. This information indicates that

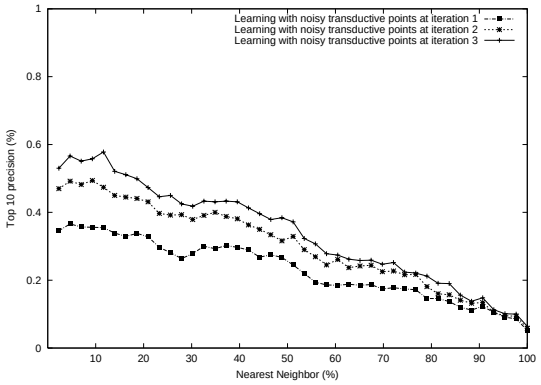


Figure 5: Negative Transduction.

using positive instances for transduction could very likely be counter-productive.

Next, we examined the effect of using negative-labeled instances to perform transduction. Even though the majority of the instances are negative for most queries (e.g., for a 1% query case, 99% of the data in the dataset are negative instances with respect to the query), using the nearest neighbors of the negative-labeled instances as transductive points has a non-zero probability of including some positive instances. Figure 5 shows that negative transduction does not seem to help. The x-axis of the figure indicates the percentage of the dataset used for transduction. For instance, at 12%, we labeled as negative 12% of the images in the dataset that are closest to the known negative instances. When fewer than 12% of the negative nearest neighbors are labeled negative, precision does not improve. When more than 12% of the negative nearest neighbors are used for transduction, the precision starts to degrade. This result indicates that 1) nearest-neighbor transduction seems too conservative to gain much useful information to improve learning performance, and 2) slight noise (i.e. the inclusion of some positive instances as negative transductive points) can make transduction ineffective.

4.3.3 Committee and Subspace Co-training

We examined the effective of the committee co-training and subspace co-training schemes. For conducting committee co-training, we collected 100 additional negative instances inferred by MEGA (i.e., additional 100 unlabeled points outside CCS) at the end of each user iteration and fed them to SVM$_{Active}$ to refine the class boundary. For conducting subspace co-training, we divided the features into 20 subsets (please refer to [13] for our feature-partition heuristics), and conducted recursive subspace training.

Figures 6(a) and (b) show the curves of precision versus iterations for 1% queries with and without co-training. Co-training indeed improves the precision for both top-10 and top-20 retrievals. On average, the committee co-training scheme achieves 5% higher precision over the pipeline scheme, and the subspace co-training scheme achieves $5-10\%$ higher precision over the committee co-training scheme. For a 1% query, the recursive subspace co-training scheme can reach 70% precision for a top-10 retrieval, and 60% for a top-20 retrieval after the fourth feedback iteration.

4.4 Observations

Our experiments have answered the four questions that we stated in the beginning of Section 4.

1. The hybrid scheme that uses MEGA to find the first relevant object(s) and then switches to SVM$_{Active}$ works significantly better than using the two learning algorithms individually.
2. When the matching data is scarce, the number of iterations required to learn a concept increases.
3. We show that co-training is a good strategy for accelerating concept convergence, whereas nearest-neighbor transductive learning does not help.

5 Measuring Perceptual Similarity

Similarity is one of the central theoretical constructs in information retrieval. To find objects (images) that are similar to a query object an information retrieval system must quantify similarity accurately and retrieve similar objects efficiently. This section addresses the central issues of *quantifying perceptual similarity* and *indexing objects in perceptual (non-metric) spaces*.

The work in the cognitive psychology community has provided ample evidence that shows similarity is both *dynamic* and *non-metric* [80, 57]. Most recently, Goldstone [28, 53] shows that similarity is the *process* that determines the respects (i.e., features, or attributes) for measuring similarity. More precisely, a similarity function for measuring a pair of objects is formulated only after the objects are compared, not before the comparison is made; and the respects for the comparison are *dynamically* activated in this formulation process. Let us use a simple example to explain. Suppose we are asked to name two places that are similar to England. Among several possibilities, Scotland and New England could be two reasonable answers. However, the respects in which England is similar to Scotland differ from those in which England is similar to New England. If we use the shared attributes of England and Scotland to compare England and New England, the latter pair might not be similar, and vice versa. Thus, a distance function using a fixed set of respects (the traditional weighted Minkowski function) cannot capture objects that are similar in different sets of respects.

Using some results of our extensive visual data mining [46], we first explain how DPF is formulated and why DPF works effectively (Section 5.1). We then discuss the link between DPF and similarity theories in cognitive psychology (Section 5.2).

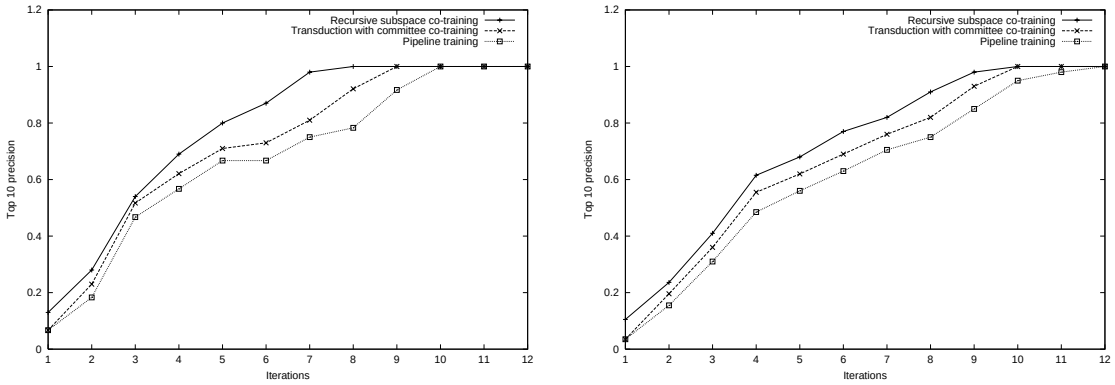


Figure 6: Precision versus iterations for (a) top-10 retrieval and (b) top-20 retrieval for 1% matching objects with and without co-training.

5.1 Visual Data Mining Results

To ensure that sound inferences could be drawn from our mining results, we carefully constructed our initial testbed as follows: First, we prepared for a dataset that was comprehensive enough to cover a diversified set of images. To achieve this goal, we collected $60,000$ JPEG images from Corel CDs and from the Internet. Second, we defined "similarity" in a slightly restrictive way so that individuals' subjectivity could be excluded. For each image in the $60,000$-image set, we performed 24 transformations (including down-sampling, cropping, rotation, and format transformation), and hence formed $60,000$ similar-image sets. The total number of images in our testbed was 1.5 million.

We examined two popular distance functions used for measuring image similarity and found some of their assumptions questionable.

- Minkowski metric. The Minkowski metric is widely used for measuring similarity between objects (e.g., images). Suppose two objects X and Y are represented by two p dimensional vectors $(x_1, x_2, \cdots, x_p)$ and $(y_1, y_2, \cdots, y_p)$, respectively. The Minkowski metric $d(X, Y)$ is defined as

$$d(X, Y) = (\sum_{i=1}^{p} |x_i - y_i|^r)^{\frac{1}{r}}, \tag{3}$$

  where r is the Minkowski factor for the norm. Particularly, when r is set as 2, it is the well known Euclidean distance; when r is 1, it is the Manhattan distance (or L_1 distance). An object located close to a query object is deemed more similar to the query object. Measuring similarity by the Minkowski metric is based on one assumption: the similar objects should be similar to the query object in all dimensions.
- Weighted Minkowski function. A variant of the Minkowski function, the weighted Minkowski distance function, has also been applied to measure image similarity. The basic idea is to apply weighting to prioritize important features. By assigning each feature a weighting coefficient w_i ($i = 1 \cdots p$), the weighted Minkowski distance function is defined as:

$$d_w(X, Y) = (\sum_{i=1}^{p} w_i |x_i - y_i|^r)^{\frac{1}{r}}. \tag{4}$$

  By applying a static weighting vector for measuring similarity, the weighted Minkowski distance function assumes that similar images resemble the query images in the same features. For example, the important features for finding a scaled image are presumed to be the same as those for finding a cropped image.

We questioned the above assumptions upon observing how similar objects are located in the feature space. To better discuss our findings, we introduce a term we have found useful in our data mining work. We define the *feature distance* on the ith feature as

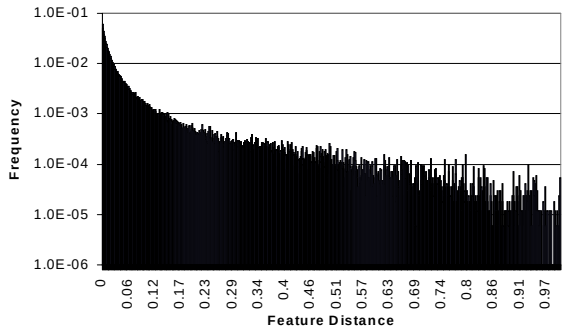
$$\delta_i = |x_i - y_i|. \qquad (i = 1, \cdots, p)$$

The expressions of Equation 3 and Equation 4 can be simplified into

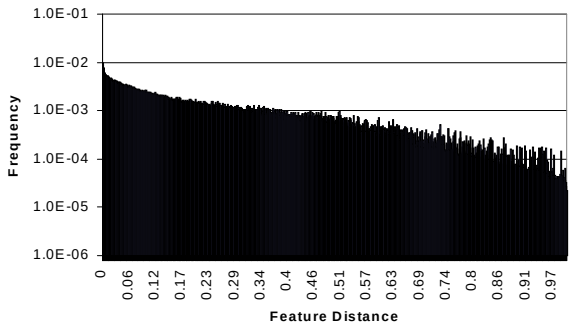
$$d(X, Y) = (\sum_{i=1}^{p} {\delta_i}^r)^{\frac{1}{r}} \quad and \quad d_w(X, Y) = (\sum_{i=1}^{p} w_i {\delta_i}^r)^{\frac{1}{r}}.$$

In our mining work, we first tallied the feature distances between similar images (denoted as δ^+), and also those between dissimilar images (denoted as δ^-). Since we normalized feature values to be between zero and one, the ranges of both δ^+ and δ^- are between zero and one. Figure 7 presents the distributions of δ^+ and δ^-. The x-axis shows the possible value of δ, from zero to one. The y-axis (in logarithmic scale) shows the percentage of the features at different δ values.

The figure shows that δ^+ and δ^- have different distribution patterns. The distribution of δ^+ is much skewed toward small values (Figure 7(a)), whereas the distribution of δ^- is more evenly distributed (Figure 7(b)). We can also see from Figure 7(a) that a moderate portion of δ^+ is in the high value range (≥ 0.5), which indicates that similar images may be quite dissimilar in many features. This observation suggests

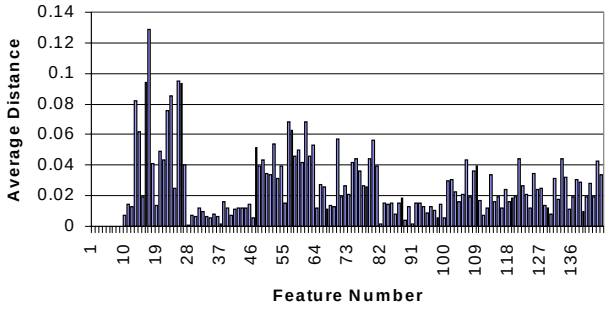


(a) Similar Images

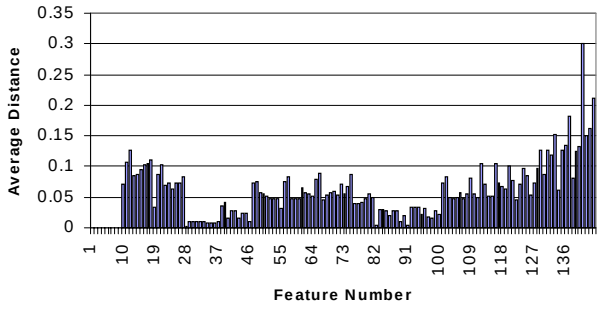


(b) Dissimilar Images

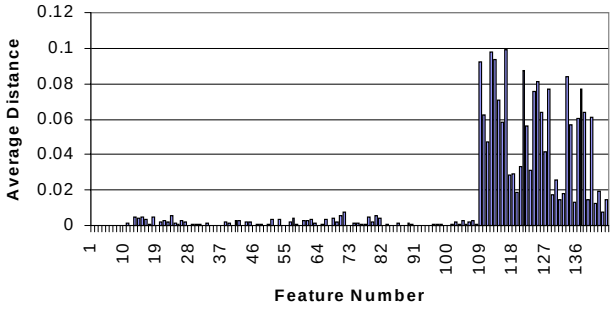
Figure 7: The Distributions of Feature Distances.



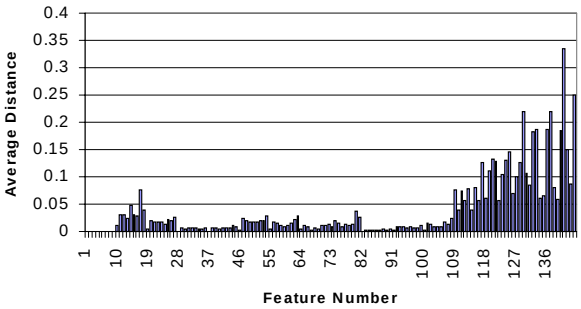
(a) GIF Images



(b) Cropped Images



(c) Rotated Images



(d) Scaled Images

Figure 8: The Average Feature Distances.

that the assumption of the Minkowski metric is inaccurate. Similar images are not necessarily similar in all features.

Next, we examined whether similar images resemble the query images in the same way. We tallied the feature distance (δ^+) of the 144 features[4] for different kinds of image transformations. Figure 8 presents four representative transformations: GIF, cropped, rotated, and scaled. The x-axis of the figure depicts the feature numbers, from 1 to 144. The first 108 features are various color features, and the last 36 are texture features. The figure shows that various similar images can resemble the query images in very different ways. GIF images have larger δ^+ in color features (the first 108 features) than in texture features (the last 36 features). In contrast, cropped images have larger δ^+ in texture features than in color features. For rotated images, the δ^+ in colors comes close to zero, although its texture feature distance is much greater. A similar pattern appears in the scaled and the rotated images. However, the magnitude of the δ^+ of scaled images is very different from that of rotated images.

We summarize our observations as follows:

1. Similar feature distance is distributed differently from dissimilar feature distance: Similar feature distance skews toward small values, whereas dissimilar feature distance shows a more even distribution.

2. Similar images do not resemble the query images in all features.

3. Images similar to the query images can be similar in differing features. Some images resemble the query image in texture, others in color. (This observation runs counter

[4] For characterizing an image, we extract a set of 144 image features. These features include color histograms, color means, color variances, color spreadness, color-blob elongation, and texture features in three orientations (vertical, horizontal, and diagonal) and three resolutions (coarse, medium, and fine) [47, 79].

to the assumption made in [88], that similar images are similar in the same way.)

The above observations not only refute the assumptions of Minkowski-type distance functions, but also provide hints as to how a good distance function would work: It does not need to consider all features equally, since similar images may match only in some features. It should weight features dynamically, since various similar images may resemble the query image in differing ways[5]. These points have led us to the design of the *dynamic partial* distance function. Let $\delta_i = |x_i - y_i|$, for $i = 1, \cdots, p$. We first define sets Δ_m as

$$\Delta_m = \{The \ \ smallest \ \ m \ \delta's \ \ of \ \ (\delta_1, ..., \delta_p)\}.$$

The first approximation of a desired perceptual function looks like this:

$$d(m, r) = (\sum_{\delta_i \in \Delta_m} {\delta_i}^r)^{\frac{1}{r}}. \quad (5)$$

5.2 Cognitive Science Interpretation

Using our experimental results, supplemented with theories and examples from cognitive psychology, we can explain why DPF works effectively as we discuss the progress of the following three similarity paradigms in cognitive psychology.

1. *Similarity is a measure of all respects* [65]. As we discussed in Section 5.1, a Minkowski-like metric accounts for all respects (features) when it is employed to measure similarity. Our mining result shown in Figure 7 is just one of a large number of counter-examples demonstrating that this assumption is questionable. The psychology studies of [53, 80] present examples showing that the Minkowski model appears to run counter to human perception of similarity.

2. *Similarity is a measure of a fixed set of respects*. Substantial work on similarity has been carried out by cognitive psychologists. The most influential work is perhaps that of Tversky [80], who suggests that similarity is determined by matching features of compared objects, and integrating these features by the formula

$$S(A, B) = \theta f(A \cap B) - \alpha f(A - B) - \beta f(B - A). \quad (6)$$

The similarity of A to B, $S(A, B)$, is expressed as a linear combination of the common and distinct features. The term $(A \cap B)$ represents the common features of A and B. $(A - B)$ represents the features that A has but B does not; $(B - A)$ represents the features that B has but A does not. The terms θ, α, and β reflect the weights given to the common and distinctive components, and function f is often assumed to be additive [53].

[5] Traditional relevance feedback methods learn a set "optimal" feature weighting for a query. For instance, if the user is more interested in color than in texture, color features are weighted higher when similarity is computed. What we have discovered here is that this "static" weighting is insufficient. An effective distance function must weight features differently when comparing the query image to different images.

The weighted Minkowski function [66] and the quadratic-form distances [25, 38] are the two representative distance functions that match the spirit of Equation 6. The weights of the distance functions can be learned via techniques such as relevance feedback [64, 66], principal component analysis, and discriminative analysis [88]. Given some similar and some dissimilar objects, the weights can be adjusted so that similar objects can be better distinguished from other objects.
However, the assumption made by these distance functions, that all similar objects are similar in the same respects [88], is questionable. As we have shown in Figure 8, GIF, cropped, rotated, and scaled images are all similar to the original images, but in differing features. For this reason, DPF allows objects to be matched to the query in different respects dynamically.

3. *Similarity is a process that provides respects for measuring similarity*. Murphy and Medin [57] provide early insights into how similarity works in human perception: "The explanatory work is on the level of determining which attributes will be selected, with similarity being at least as much a consequence as a cause of a concept coherence." The result presented in Figure 8 confirms this statement. Just as in human perception, which sees similarities after seeing the comparable objects, a distance function for measuring a pair of objects is formulated only after the objects are compared, not before the comparison is made. The respects for the comparison are activated in this formulation process. The activated respects are more likely to be those that can support coherence between the compared objects.

With those paradigms in mind, let us re-examine how DPF works. DPF activates different features for different object pairs. The activated features are those with minimum differences—those which provide coherence between the objects. If coherence can be maintained (because a sufficient number of features are similar), then the objects paired are perceived as similar.

6 Empirical Study II — Distance Functions

We conducted an empirical study to examine the effectiveness of DPF. Our experiments consisted of two parts.

1. We compared DPF with the Euclidean distance function and L_1 distance function, the most widely used similarity functions in image retrieval. We also compared DPF with the fractional distance function, which has recently been shown more effective than L_2 and L_1 distance functions [2].

2. In addition to the unweighted versions, we also examined whether the weighted DPF is effective for enhancing the performance of the weighted Minkowski distance function (Section 6.3).

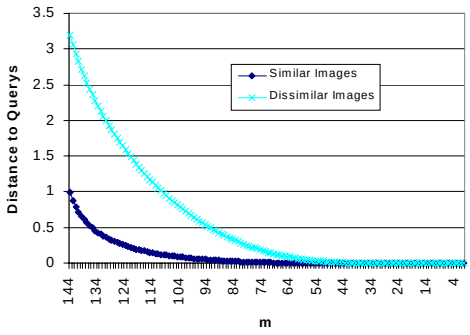
Our empirical study of image retrieval consisted of two parts: training and testing. In the training part, we used the 1.5M-image dataset to predict the optimal m value for DPF. In the testing part, we set DPF with the optimal m value,

and tested it on an independently constructed 50K-image dataset to examine its effectiveness.

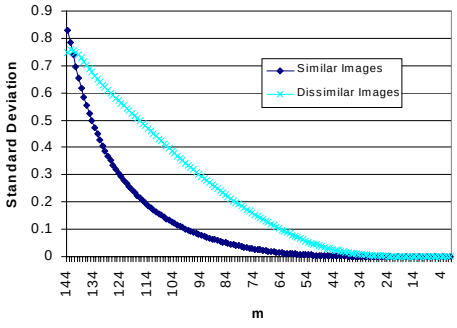
6.1 Predicting m Through Training

The design goal of DPF is to better separate similar images from dissimilar ones. To meet this design goal, we must judiciously select parameter m. (We take the Euclidean distance function as the baseline, thus we set $r = 2$ for both DPF and the Minkowski distance function.)

To find the optimal m value, we used the $60,000$ original images to perform queries. we applied DPF of different m values to the 1.5M-image dataset. The 24 images with the shortest distance from each query image were retrieved. For each of the five similar-image categories (i.e., GIF, cropped, downsampled, rotated, or scaled), we observed how many of them failed to appear in the top-24 results. Figure 9 presents the average rate of missed images for each similar-image category. The figure shows that when m is reduced from 144 to between 110 and 118, the rates of missing are near their minimum for all five similar-image categories. (Note that when $m = 144$, DPF degenerates into the Euclidean function.) DPF outperforms the Euclidean distance function by significant margins for all similar-image categories.



(a) Average of Distances



(b) Standard Deviation of Distances

Figure 10: The Effect of DPF.

To investigate why DPF works effectively when m is reduced, we tallied the distances from these $60,000$ queries to their similar images and their dissimilar images, respectively. We then computed the average and the standard deviation of these distances. We denote the average distance of the similar images to their queries as μ_d^+, of the dissimilar images as μ_d^-. We denote the standard deviation of the similar images' distances as σ_d^+, of the dissimilar images as σ_d^-.

Figure 10 depicts the effect of m (in the x-axis) on μ_d^+, μ_d^-, σ_d^+, and σ_d^-. Figure 10(a) shows that as m becomes smaller, both μ_d^+ and μ_d^- decrease. The average distance of similar images (μ_d^+), however, decreases at a faster pace than that of dissimilar images (μ_d^-). For instance, when we decrease m from 144 to 130, μ_d^+ decreases from 1.0 to about 0.3, a 70% decrease, whereas μ_d^- decreases from 3.2 to about 2.0, a 38% decrease. This gap indicates μ_d^+ is more sensitive to the m value than μ_d^-. Figure 10(b) shows that the standard deviations σ_d^+ and σ_d^- observe the same trend as the average distances do. When m decreases, similar images become more compact in the feature space, at a faster pace than dissimilar images do.

To provide more detailed information, Figure 11 depicts the distance distributions at four different m values. Figure 11(a) shows that when $m = 144$, a significant overlap occurs between the distance distributions of similar and dissimilar images to the query images. (When $m = 144$, DPF degenerates to the Euclidean function.) In other words, many similar images and dissimilar images may reside about the same distance from their query image, which causes degraded search performance. When we decrease m to 124, Figure 11(b) shows that both distributions shift toward the left. The distribution of similar images becomes more compact, and this leads to a better separation from dissimilar images. Further decreasing the m value moves both distributions leftward (as shown in Figures 11(c) and 11(d)). When little room is left for the distance distribution of similar images to move leftward, the overlap can eventually increase. Our observations from these figures confirm that we need to find the optimal m value to achieve best separation for similar and dissimilar images.

6.2 Testing DPF

We tested our distance functions on a dataset that was independently constructed from the 1.5M-image dataset used for conducting mining and parameter training.

The test dataset consisted of 50K randomly collected World Wide Web images. Among these images we identified 100 images as query images. For each query image, we generated 24 similar images using the transformation methods described in Section 5.1. We also visually identified 5 similar images for each query image. (See Figure 12 for examples of visually-identified similar images).

We conducted 100 queries using the 100 query images. For each query, we recorded the distance ranks of its similar images. For DPF, we fixed m value as 114 based on the training results in Section 6.1. Figure 13 depicts the experimental results. The precision-recall curves in the figure show that the search performance of DPF is significantly better than the other traditional distance functions. For instance, to achieve a recall of 80%, the retrieval precision of DPF is 84%, whereas the precision of the L_1 distance and

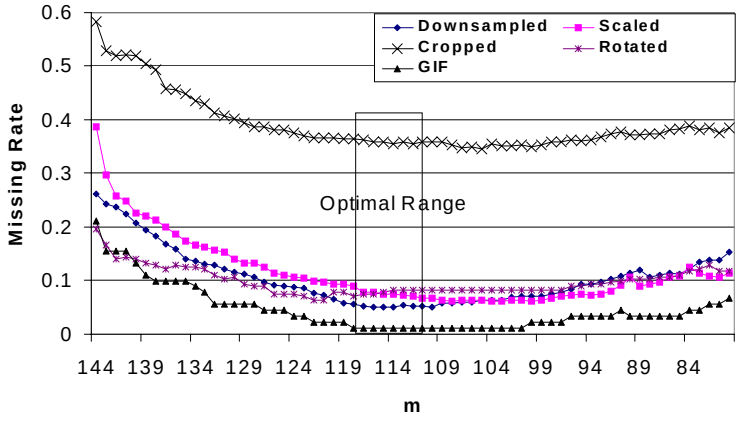
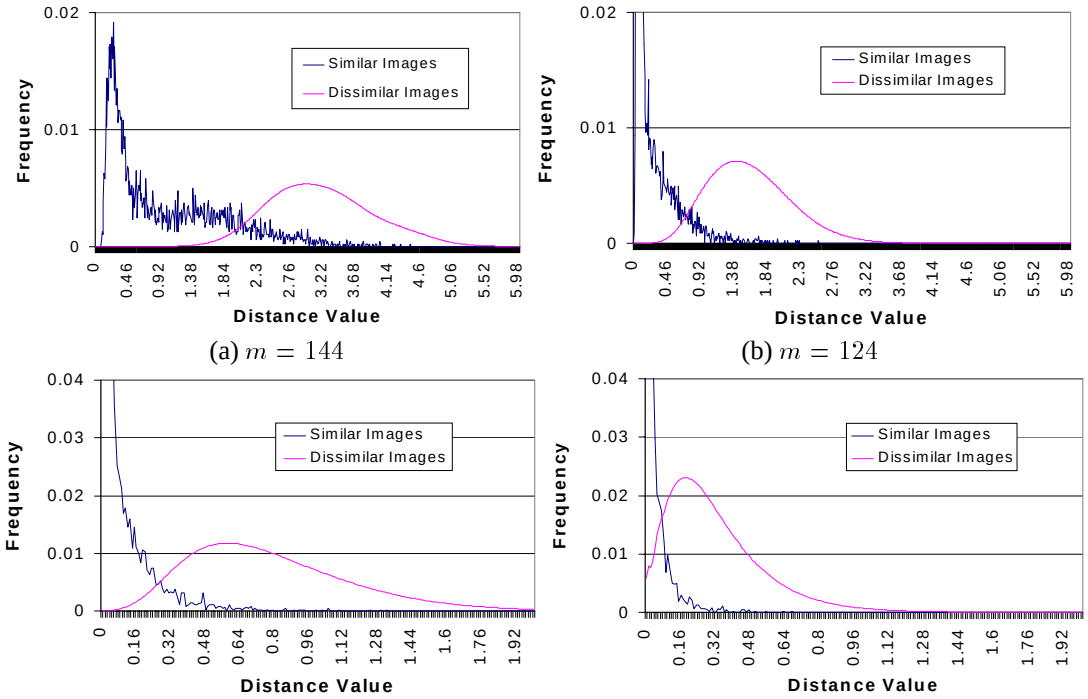


Figure 9: Training for the Optimal m Value.



(a) $m = 144$ (b) $m = 124$

(c) $m = 104$ (d) $m = 84$

Figure 11: Distance Distributions vs. m.

the Euclidean distance is 70% and 50%, respectively (Figure 13). DPF also works consistently better than the best setting, $r = 0.5$, of the fractional function.

We were particularly interested in the retrieval performance of the visually identified similar images, which were not included within the training-image dataset. Figure 14 compares the retrieval performance of DPF and traditional distances for the visually identified similar images. The precision-recall curves indicate that, even though the visually identified similar images were not included in the training-image dataset, DPF could still find them most effectively in the testing phase. This indicates that the trained DPF parameters can be generalized to find similar images produced by methods other than those for producing the training dataset.

6.3 Weighted DPF vs. Weighted Euclidean

We were also interested in applying weighted DPF to improve the weighted Minkowski distance function, which has been used extensively to personalize similarity measures. For weighted Minkowski distance, a weighting vector is learned for each query. Usually, the weight of a feature is set as the inverse of the variance of its values among similar images. Here, we allowed the weighted Euclidean distance function to work under the ideal condition—that is, it knows



Figure 12: A Query Image (The Upper Left One) and Five Visually Identified Similar Images.

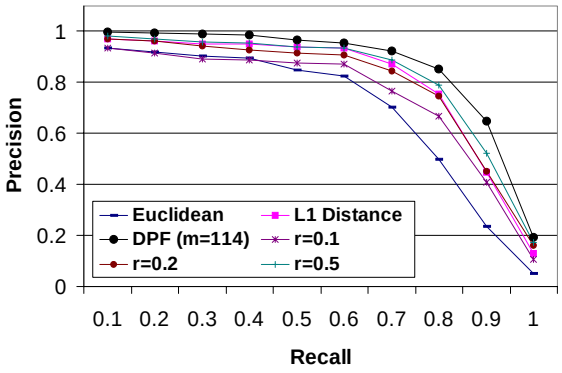


Figure 13: Precision/Recall for Similar Images.

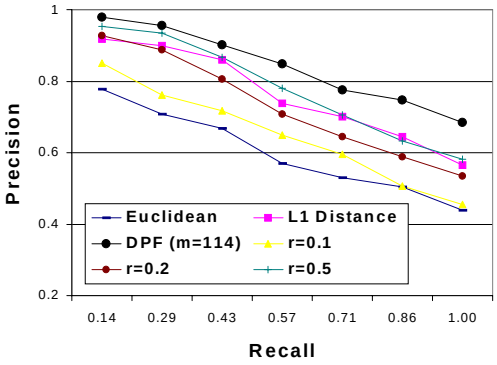


Figure 14: Precision/Recall for Visually Similar Images.

all similar images a priori and can compute the ideal weighting vector for each query. Figure 15 shows that the weighted Euclidean function outperforms its unweighted counterpart. This result confirms that the weighted version [64, 58] is indeed a better choice than the unweighted version (provided that the appropriate weighting can be learned). However, there is still much room for improvement. When we applied weighted DPF using the same weighting vector, its retrieval performance was better than that of the weighted Euclidean distance function. For instance, at 80% recall rate, the retrieval precision of the weighted Euclidean distance is about 68%, whereas the weighted DPF could achieve a precision of above 85%.

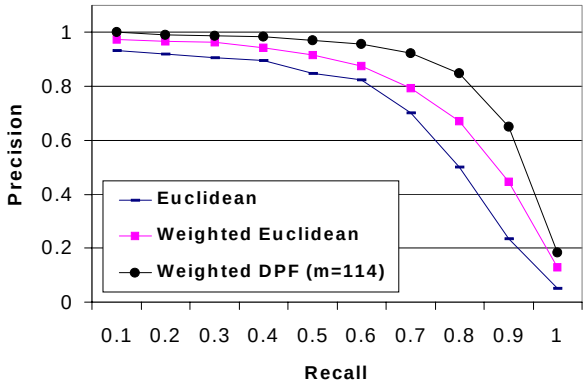


Figure 15: Comparision of Weighted Functions.

6.4 Observations

We summarize the results of our experiments as follows:

1. DPF is more effective than some most representative distance functions used in the CBIR community (e.g., Minkowski-like functions) for measuring image similarity and for detecting shot transitions.

2. The weighted version of DPF outperforms the weighted version of the Euclidean distance function.

3. We believe that DPF can be generalized to find similar images of some other ways, and that DPF can be effective when a different set of low-level features are employed. Our belief is partially supported by our empirical results, and partially justified by similar theories in cognitive science, which we discuss next.

7 Concluding Remarks

We have depicted methods for learning subjective query concepts and measuring perceptual similarity, and presented encouraging results. The work can be further enhanced in at least three directions.

1. *Learning method enhancement*. We believe that the *subspace co-training* method depicted in Section 3.2.3 can be improved by judiciously selecting most useful subspaces. Recent works in the database community [2, 1], in statistics (e.g., random forests [7]), and our own DPF function all indicate that carefully chosen subspaces may provide different and even enhanced views on the data, and aggregating these different views can improve the quality of statistical inference.

2. *Similarity measurement improvement*. DPF is effective; however, selecting a good m (i.e., a good subspace that consists of m relevant features) for each pair of images can be difficult. (Currently, we train an m that works the best "on the average" for the entire image set.) According to our empirical results, different similar image-pairs are best captured at different m settings (e.g., some at $m = 94$, and some at $m = 114$) and the variance of these m's can be large. By using an average m, we can miss retrieving some images that are obviously similar. To alleviate this

large variance problem, we have been experimenting with a multi-resolution version of DPF. We first divide image features into multiple resolutions. Our similarity measure starts at the coarsest features (e.g., color histograms), and then proceeds to finer levels (e.g., color shapes and spreadness). Our preliminary result shows this method promising, since the variance at the coarsest feature level is quantitatively much smaller.

3. *Context-based similarity measurement*. Another challenge is combining a learned query concept with DPF and making DPF context-dependent. For instance, a query concept of "tigers" may require certain color and texture features to be present in a matching image. The learner must convey contextual information to DPF so that useful features are not mistakenly removed. We believe that context-based DPF can be formulated by taking advantage of the hyperplane generated by SVM_{Active} that separates relevant from irrelevant images. The orientation of the hyperplane in the projected space can be back-projected to the input space and context-based feature weightings then computed. We need to further study the best method for selecting a good feature subspace to combine perceptual similar channels with the contextual information.

We would like to offer some speculative thoughts about how our work might fit within the grander vision of database research in the next ten years. Let D denote the number of data dimensions, N the number of data vectors (i.e., tuples), $d(X, Y)$ the distance function that computes similarity between vectors X and Y, and $\overline{LC}$ the average database-literacy of endusers. The broader impact of furthering this work may be its potential contributions to the following research challenges:

1. *Decreasing $\overline{LC}$ and increasing demands for personalization support*. Underprivileged, young, and database-illiterate users who are not trained to formulate their needs in SQL and XML query languages cannot enjoy the full benefits of the information revolution. Making information more accessible and more personalized should be one of the missions of the database community. Intelligent query-concept learners will make text-based and image-based information retrieval easier and more personalized. A text and/or image collection can employ an intelligent learner to better capture users' query concepts. This way, even non-technical people who have not learned Boolean algebra can access information through a concept learner, which helps the user formulate and refine query statements.

2. *Dynamic and non-metric $d(X, Y)$*. If we are convinced that context-based DPF or the fractional function are more appropriate for measuring subjective perceptual similarity, we are at once confronted by the challenges of their dynamic and non-metric properties. For instance, traditional indexing schemes that require metric-space properties [83, 48] cannot work with DPF. Furthermore, since the *respects* for measuring similarity by DPF are determined in a pairwise fashion, or the feature space of different object-pairs can be different, coordinate-based indexing structures (e.g., tree-like structures [15, 43], hash-based structures [37, 36]) cannot index DPF either. New indexing structures must be designed to work with these distance functions.

3. $D > N$. Emerging data analysis in e.g., biotech data and imagery faces the $D > N$ high dimensionality challenge. As pointed by D. Donoho [23], the theories underlying "classical" data analysis are based on the assumption of $D < N$, and $N \to \infty$, but when $D > N$, the basic methodology which was used in the classical world is not strictly applicable. The statistical inference and database research under the $D > N$ constraint that our colleagues and we have embarked on barely scratches the surface of the problem.

We close this paper with David Donoho's forward-looking statement [23] in high-dimensional data analysis.

"We can say with complete confidence that in the coming century, high-dimensional data analysis will be a very significant activity, and completely new methods of high-dimensional data analysis will be developed; we just don't know what they are yet."

Acknowledgements

This project is partially supported by NSF grants IIS-0133802 (Career) and IIS-0219885 (ITR). The first author is supported by an IBM Faculty Partnership Award.

References

[1] C. C. Aggarwal. On the effects of dimensionality reduction on high dimensional similarity search. *ACM PODS*, 2001.

[2] C. C. Aggarwal, A. Hinneburg, and D. A. Keim. On the surprising behavior of dstance metrics in high dimensional space. *ICDT*, 2001.

[3] C. Bishop. *Neural Networks for Pattern Recognition*. Oxford, 1998.

[4] A. Blum and T. Mitchell. *Combining Labeled and Unlabeled Data wih Co-Training*. Proceedings of the Workshop on Computational Learning Theory, 1998.

[5] L. Breiman. Bagging predicators. *Machine Learning*, pages 123–140, 1996.

[6] L. Breiman. Arcing classifiers. *The Annals of Statistics*, pages 801–849, 1998.

[7] L. Breiman. Random forests. *UC Berkeley Technical Report*, 2001.

[8] C. Burges. A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery*, 2:121–167, 1998.

[9] C. Carson, S. Belongie, H. Greenspan, and J. Malik. Region-based image querying. *Proc. of IEEE Workshop on Content-based Access of Image and Video Libraries, in conjunction with IEEE CVPR*, 1997.

[10] N. Cesa-Bianchi, Y. Freund, D. Haussler, D. P. Helmbold, R. E. Schapire, and M. K. Warmuth. How to use expert advice. *Journal of ACM*, 44(3):427–485, 1997.

[11] E. Chang, K.-T. Cheng, and L. Chang. PBIR — perception-based image retrieval. *ACM Sigmod (Demo)*, May 2001.

[12] E. Chang, K.-T. Cheng, W.-C. Lai, C.-T. Wu, C.-W. Chang, and Y.-L. Wu. PBIR — a system that learns subjective image query concepts. *Proceedings of ACM Multimedia, http://www.mmdb.ece.ucsb.edu/˜demo/corelacm/*, pages 611–614, October 2001.

[13] E. Chang and B. Li. Mega — the maximizing expected generalization algorithm for learning complex query concepts (extended version). *Technical Report http://www-db.stanford.edu/∼echang/mega.pdf*, November 2000.

[14] S. Chaudhuri and L. Gravano. Evaluating top-k selection queries. *VLDB*, 1999.

[15] P. Ciaccia, M. Patella, and P. Zezula. M-Tree: An efficient access method for similarity search in metric spaces. *Proceedings of the 23rd VLDB*, August 1997.

[16] D. Cohn, Z. Ghahramani, and M. Jordan. Active learning with statistical models. *Journal of Artificial Intelligence Research*, 4, 1996.

[17] I. Cox, J. Ghosn, M. Miller, T. Papathomas, and P. Yianilos. Introduces pichunter, the bayesian framework, and describes a working system including measured user performance. hidden annotation in content based image retrieval. In *IEEE Workshop on Content-Based Access of Image & Video Libraries*, pages 76–81, June 1997.

[18] I. Cox, M. Miller, S. Omohundo, and P. Yianilos. Pichunter: Bayesian relevance feedback for image retrieval. In *13th International Conference on Pattern Recognition*, pages 361–369, August 1996.

[19] I. Cox, M. Miller, S. Omohundo, and P. Yianilos. Target testing and the pichunter bayesian multimedia retrieval system. In *Proceedings of the Forum on Research & Technology Advances in Digital Libraries*, pages 66–75, 1996.

[20] I. J. Cox, M. L. Miller, T. P. Minka, T. V. Papathomas, and P. N. Yianilos. The bayesian image retrieval system, pichunter: Theory, implementation and psychological experiments. *IEEE Transaction on Image Processing (to appear)*, 2000.

[21] I. Dagan and S. Engelson. Committee-based sampling for training probabilistic classifiers. In *Proceedings of the Twelfth International Conference on Machine Learning*, pages 150–157. Morgan Kaufmann, 1995.

[22] T. Dietterich and G. Bakiri. Solving multiclass learning problems via error-correcting output codes. *Journal of Artifical Intelligence Research*, 2, 1995.

[23] D. L. Donoho. Aide-memoire. high-dimensional data analysis: The curses and blessings of dimensionality. *American Math. Society Lecture — Math Challenges of the 21st Century*, 2000.

[24] S. Dumais, J. Platt, D. Heckerman, and M. Sahami. Inductive learning algorithms and representations for text categorization. In *Proceedings of the Seventh International Conference on Information and Knowledge Management*. ACM Press, 1998.

[25] M. Flickner, H. Sawhney, J. Ashley, Q. Huang, B. Dom, M. Gorkani, J. Hafner, D. Lee, D. Petkovic, D. Steele, and P. Yanker. Query by image and video content: the QBIC system. *IEEE Computer*, 28(9):23–32, 1995.

[26] Y. Freund, H. Seung, E. Shamir, and N. Tishby. Selective sampling using the Query by Committee algorithm. *Machine Learning*, 28:133–168, 1997.

[27] E. B. Goldstein. *Sensation and Perception (5th Edition)*. Brooks/Cole, 1999.

[28] R. L. Goldstone. Similarity, interactive activation, and mapping. *Journal of Experimental Psychology: Learning, Memory, and Cog nition*, 20:3–28, 1994.

[29] T. Graepel, R. Herbrich, and K. Obermayer. Bayesian transduction. *Advances in Neural Information Systems Processing*, 12, 2000.

[30] A. Grove and D. Schuurmans. Boosting in the limit: Maximizing the margin of learned ensembles. In *Proc. 15th National Conference on Artificial Intelligence (AAAI)*, 1998.

[31] A. Gupta and R. Jain. Visual information retrieval. *Comm. of the ACM*, 40(5):69–79, 1997.

[32] W. Hsu, T. S. Chua, and H. K. Pung. An integrated color-spatial approach to content-based image retrieval. *ACM Multimedia Conference*, pages 305–313, 1995.

[33] M. Hu. Visual pattern recognition by moment invariants. *IRE Trans. Information Theory ,IT-8, 2*, pages 179–187, 1962.

[34] K. A. Hua, K. Vu, and J.-H. Oh. Sammatch: A flexible and efficient sampling-based image retrieval technique for image databases. *Proceedings of ACM Multimedia*, November 1999.

[35] J. Huang, S. R. Smith, M. Mitra, W.-J. Zhu, and R. Zabih. Image indexing using color correlograms. *Proc. IEEE Comp. Soc. Conf. Comp. Vis. and Patt. Rec.*, pages 762–768, 1997.

[36] P. Indyk, A. Gionis, and R. Motwani. Similarity search in high dimensions via hashing. *Proceedings of the 25th VLDB*, September 1999.

[37] P. Indyk and R. Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. *Proceedings of the 30th STOC*, pages 604–13, 1998.

[38] Y. Ishikawa, R. Subramanya, and C. Faloutsos. Mindreader: Querying databases through multiple examples. *VLDB*, 1998.

[39] G. James and T. Hastie. *Error Coding and PaCT's*. 1997.

[40] T. Joachims. Text categorization with support vector machines. In *Proceedings of the European Conference on Machine Learning*. Springer-Verlag, 1998.

[41] T. Joachims. Transductive inference for text classification using support vector machines. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 200–209. Morgan Kaufmann, 1999.

[42] K. S. Jones and P. W. (Editors). *Readings in Information Retrieval*. Morgan Kaufman, July 1997.

[43] N. Katayama and S. Satoh. The SR-Tree: An index structure for high-dimensional nearest neighbor queries. *Proceedings of ACM SIGMOD*, May 1997.

[44] M. Kearns, M. Li, and L. Valiant. Learning boolean formulae. *Journal of ACM*, 41(6):1298–1328, 1994.

[45] M. Kearns and U. Vazirani. *An Introduction to Computational Learning Theory*. MIT Press, 1994.

[46] B. Li and E. Chang. Discovery of a perceptual distance function for measuring image similarity. *ACM Multimedia Journal Special Issue (to appear)*, 2002.

[47] B. Li, E. Chang, and C.-S. Li. Learning image query concepts via intelligent sampling. *Proceedings of IEEE Multimedia and Expo*, August 2001.

[48] C. Li, E. Chang, H. Garcia-Molina, and G. Wiederhold. Clustering for approximate similarity queries in high-dimensional spaces. *IEEE Transaction on Knowledge and Data Engineering (to appear)*, 2001.

[49] J. Li, J. Z. Wang, and G. Wiederhold. Irm: Integrated region matching for image retrieval. *Proceedings of ACM Multimedia*, October 2000.

[50] B. Manjunath, P. Wu, S. Newsam, and H. Shin. A texture descriptor for browsing and similarity retrieval. *Signal Processing Image Communication*, 2001.

[51] B. S. Manjunath and W. Y. Ma. Texture features for browsing and retrieval of image data. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 18(8):837–842, August 1996.

[52] A. McCallum and K. Nigam. Employing EM in pool-based active learning for text classification. In *Proceedings of the Fifteenth International Conference on Machine Learning*. Morgan Kaufmann, 1998.

[53] D. L. Medin, R. L. Goldstone, and D. Gentner. Respects for similarity. *Psychological Review*, 100(2):254–278, 1993.

[54] T. Michell. *Machine Learning*. McGraw Hill, 1997.

[55] T. M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.

[56] M. Moreira and E. Mayoraz. Improving pairwise coupling classification with error correcting classifiers. *Proceedings of the Tenth European Conference on Machine Learning*, April 1998.

[57] G. Murphy and D. Medin. The role of theories in conceptual coherence. *Psychological Review*, 92:289–316, 1985.

[58] M. Ortega, Y. Rui, K. Chakrabarti, S. Mehrotra, and T. S. Huang. Supporting similarity queries in mars. *Proc. of ACM Conf. on Multimedia*, 1997.

[59] M. Ortega, Y. Rui, K. Chakrabarti, A. Warshavsky, S. Mehrotra, and T. S. Huang. Supporting ranked boolean similarity queries in mars. *IEEE Transaction on Knowledge and Data Engineering*, 10(6):905–925, December 1999.

[60] C. Papageorgiou, M. Oren, and T. Poggio. A general framework for object detection. In *Proceedings of the International Conference on Computer Vision*, 1998.

[61] G. Pass and R. Zabih. Histogram refinement for content-based image retrieval. *IEEE Workshop on Applications of Computer Vision*, pages 96–102, 1996.

[62] G. Pass, R. Zabih, and J. Miller. Comparing images using color coherence vectors. *Proceedings of the Fourth ACM Multimedia Conference*, pages 65–73, 1996.

[63] K. Porkaew, K. Chakrabarti, and S. Mehrotra. Query refinement for multimedia similarity retrieval in mars. *Proceedings of ACM Multimedia*, November 1999.

[64] K. Porkaew, S. Mehrota, and M. Ortega. Query reformulation for content based multimedia retrieval in mars. *ICMCS*, pages 747–751, 1999.

[65] M. W. Richardson. Multidimensional psychophysics. *Psycholigical Bulletin*, 35:659–660, 1938.

[66] J. Rocchio. Relevance feedback in information retrieval. In G. Salton, editor, *The SMART retrival system: Experiments in automatic document processing*. Prentice-Hall, 1971.

[67] Y. Rubner, C. Tomasi, and L. Guibas. Adaptive color-image embedding for database navigation. *Proceedings of the the Asian Conference on Computer Vision*, January 1998.

[68] Y. Rui and T. Huang. Optimizing learning in image retrieval. *Proceedings of IEEE Computer Vision and Pattern Recognition*, pages 236–243, 2000.

[69] Y. Rui, T. S. Huang, and S.-F. Chang. Image retrieval: Current techniques, promising directions and open issues. *Journal of Visual Communication and Image Representation*, March 1999.

[70] Y. Rui, T. S. Huang, M. Ortega, and S. Mehrotra. Relevance feedback: A power tool in interactive content-based image retrieval. *IEEE Tran on Circuits and Systems for Video Technology*, 8(5), Sept 1998.

[71] R. Schapire, Y. Freund, P. Bartlett, and W. Lee. Boosting the margin: A new explanation for the effectiveness of voting methods. In *Proceeding of the Fourteenth International Conference on Machine Learning*. Morgan Kaufmann, 1997.

[72] H. Seung, M. Opper, and H. Sompolinsky. Query by committee. In *Proceedings of the Fifth Workshop on Computational Learning Theory*, pages 287–294. Morgan Kaufmann, 1992.

[73] J. Smith and S.-F. Chang. Automated image retrieval using color and texture. *IEEE Transaction on Pattern Analysis and Machine Intelligence*, November 1996.

[74] J. Smith and S.-F. Chang. Tools and techniques for color image retrieval. *Volume 2670 of SPIE Proceedings*, pages 1630–1639, 1996.

[75] J. R. Smith and S.-F. Chang. Visualseek: A fully automated content-based image query system. *ACM Multimedia Conference*, 1996.

[76] M. Stricker and M.Orengo. Similarity of color images. *SPIE Conference on Storage and Retrieval for Image and Video Dat abase III*, pages 381–92, February 1995.

[77] M. Swain and D. H. Ballard. Color indexing. *Int. Journal of Computer Vision*, 7(1):11–32, 1991.

[78] H. Tamura, S. Mori, and T. Yamawaki. Texture features corresponding to visual perception. *IEEE Transaction on Systems Man Cybernet (SMC)*, 1978.

[79] S. Tong and E. Chang. Support vector machine active learning for image retrieval. *Proceedings of ACM International Conference on Multimedia*, pages 107–118, October 2001.

[80] A. Tversky. Feature of similarity. *Psychological Review*, 84:327–352, 1977.

[81] V. Vapnik. *Statistical Learning Theory*. Wiley, 1998.

[82] B. Wandell. *Foundations of Vision*. Sinauer, 1995.

[83] J. Z. Wang, J. Li, and G. Wiederhold. Wise: A wavelet-based image search engine with efficient feature vector clustering and classification. *Submitted for journal publication*, 1998.

[84] J. Z. Wang, G. Wiederhold, O. Firschein, and S. X. Wei. Wavelet-based image indexing techniques with partial sketch retrieval capability. *Proceedings of the 4th ADL*, May 1997.

[85] J. Z. Wang, G. Wiederhold, O. Firschein, and S. X. Wei. Content-based image indexing and searching using daubechies' wavelets. *Journal of Digital Libraries*, 1(4):311–28, 1998.

[86] L. Wu, C. Faloutsos, K. Sycara, and T. R. Payne. Falcon: Feedback adaptive loop for content-based retrieval. *The* 26th *VLDB Conference*, September 2000.

[87] O. R. Zaiane, J. Han, Z. N. Li, J. Y. Chiang, and S. Chee. Multimedia-miner: A system prototype for multimedia data mining. *ACM Sigmod (System Demo)*, June 1998.

[88] X. S. Zhou and T. S. Huang. Comparing discriminating transformations and svm for learning during multimedia retrieval. *Proc. of ACM Conf. on Multimedia*, pages 137–146, 2001.

Appendix: Example Figures

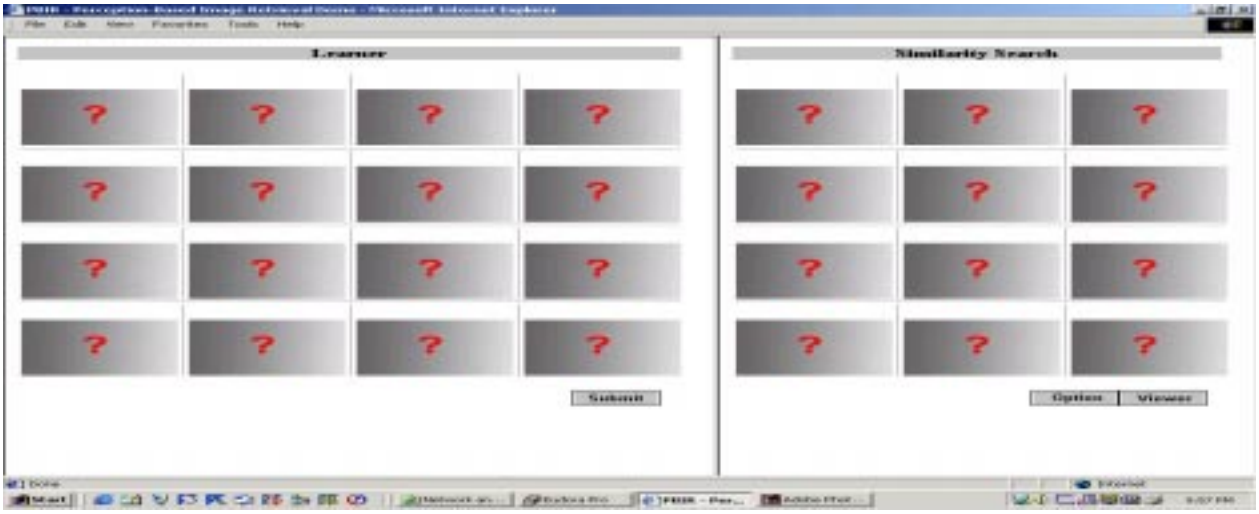


Figure 16: Initial Screen.

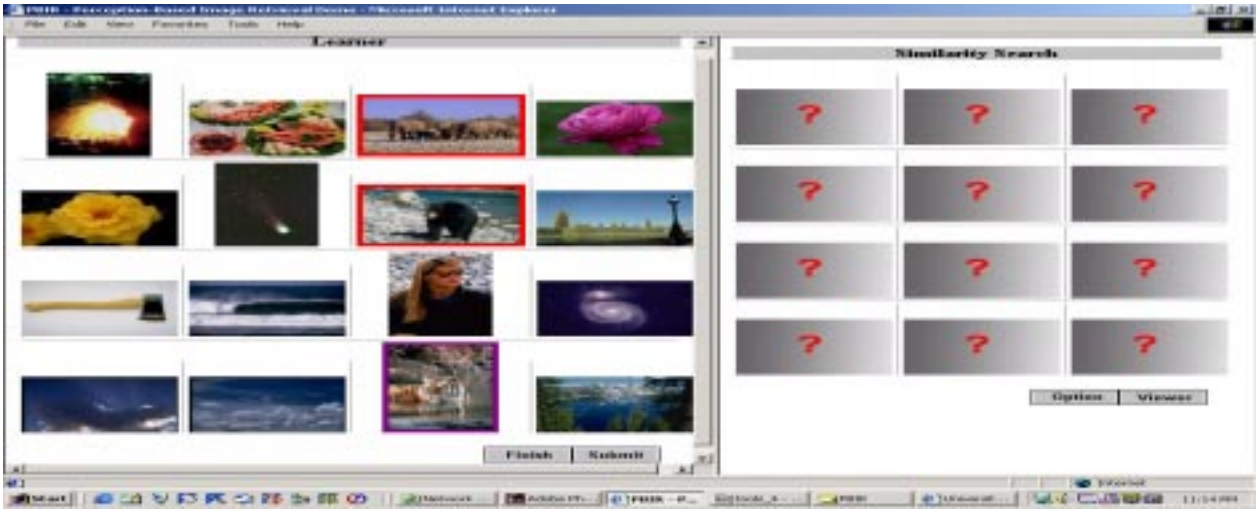


Figure 17: Wild Animal Query Iteration #1.

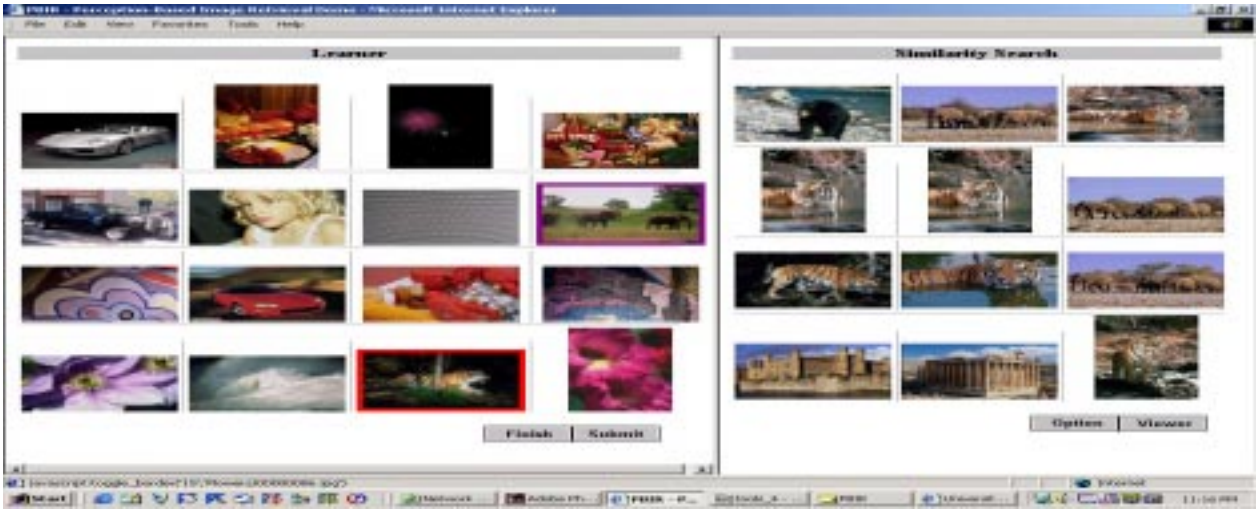


Figure 18: Wild Animal Query Iteration #2.



Figure 19: Wild Animal Query Iteration #3.



Figure 20: Wild Animal Query Final Result.